

1. Identificación de Pares: Enumera tres claves (keys) presentes en el nivel superior del objeto JSON.

```
"hostname": "R1-Core",  
"uptime_days": 45,  
"managed_by_apic": true
```

2. Tipos de Datos: Identifica un ejemplo de cada uno de los siguientes tipos en el código: String, Number, Boolean y Null.

String: "R1-Core""GigabitEthernet0/0"

Number: 45 o 1500

Boolean: true

Null: null

Parte 2: Detección de Errores de Sintaxis

En la automatización de redes, un error de sintaxis en un payload JSON puede causar que la configuración de un dispositivo falle. Analiza el siguiente fragmento de código destinado a configurar una VLAN y responde.

```
{  
  "vlan_id": 10  
  "name": "Usuarios_Ventas"  
  "tagged": true,  
  "description": "VLAN para el departamento de ventas",  
}
```

3. ¿Qué errores de sintaxis impiden que este JSON sea válido según los estándares de red y programación?

Selecciona las dos opciones correctas.

A) El valor de 'vlan_id' debería estar entre comillas dobles obligatoriamente.

B) Las claves como 'vlan_id' no deben llevar guiones bajos.

C) Existe una coma final (trailing comma) después del valor de 'description'.

D) Faltan comas para separar los pares clave-valor después de 'vlan_id' y 'name'.

4. Reflexión Técnica: Explica por qué es crítico que un controlador como Cisco DNA Center reciba un JSON perfectamente validado antes de aplicar cambios en la red.

Parte 2: Comparativa de Formatos (JSON vs. XML)

En redes modernas, REST APIs suelen preferir JSON por ser más ligero, pero muchos sistemas heredados o protocolos como NETCONF utilizan XML. Observa cómo se estructuran los mismos datos en ambos formatos.

Formato JSON	Formato XML
<pre>{ "hostname": "R1", "status": "up" }</pre>	<pre>xml <device> <hostname>R1</hostname> <status>up</status> </device></pre>

Comparativa JSON vs XML

JSON

Utiliza llaves y comillas ("clave": "valor").

No necesita etiquetas de cierre.

XML

Etiquetas de apertura y cierre (<tag>...</tag>).

Más como texto.